Deep Learning 2025
Assignment 1

Gustav Engqvist (*npm871*) & Lasse Strandbygaard (*nhs881*)

November 28, 2025

1 Backpropagation

1.1 c

c) Write the equations for an FFNN with L layers as recursion. Use l as the index for the layer:

$$y_j = h_L(a_j^{(L)}) \tag{1}$$

$$z_j^{(l)} = h_l(a_j^{(l)}) \qquad l = 1, \ldots, L \tag{2}$$

$$a_j^{(l)} = \sum_i^M w_{ji}^{(l)} z_i^{(l-1)} \qquad l = 2, \ldots, L \tag{3}$$

$$a_j^{(1)} = \sum_{i=0}^D w_{ji}^{(1)} x_i \tag{4}$$

1.2 j

We use that the model is layered so the chain-rule we need is:

$$\frac{\partial E(w)}{\partial w_{ji}^{(l)}} \sum_{k=1}^K \frac{\partial E(w)}{\partial a_k^{(l+1)}} \frac{\partial a_k^{(l+1)}}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial w_{ji}^{(l)}}$$

We can replace the first term with δ_k so it becomes:

$$\frac{\partial E(w)}{\partial w_{ji}^{(l)}} \sum_{k=1}^K \delta_k^{(l+1)} \frac{\partial a_k^{(l+1)}}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial w_{ji}^{(l)}}$$

Now we can compute the partial derivatives:

$$\frac{\partial a_j^{(l)}}{\partial w_{ji}^{(l)}} = z_i^{(l-1)}$$

$$\frac{\partial a_k^{(l+1)}}{\partial a_j^{(l)}} = w_{kj}^{(l+1)} h_l'(a_j^{(l)})$$

Now we can insert the derivatives:

$$\frac{\partial E(w)}{\partial w_{ji}^{(l)}} \sum_{k=1}^{K} \delta_k^{(l+1)} w_{kj}^{(l+1)} h_l'(a_j^{(l)}) z_i^{(l-1)}$$

We will then define the parts that are not depended on i:

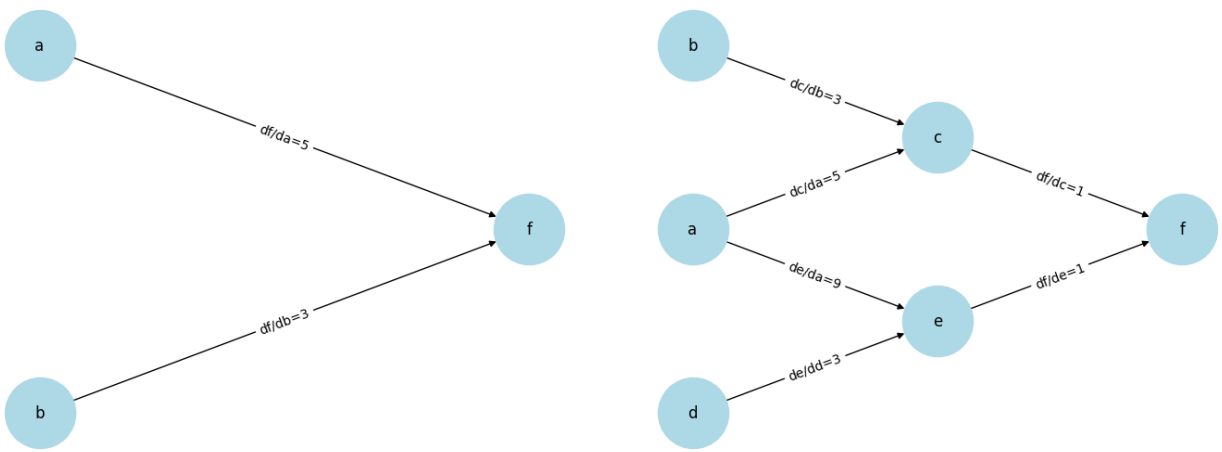
$$\delta_j^{(l)} = \sum_{k=1}^{K} \delta_k^{(l+1)} w_{kj}^{(l+1)} h_l'(a_j^{(l)})$$

And then we can instert this into the general backpropagation rule:

$$\frac{\partial E(w)}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} z_i^{(l-1)}$$

2 AutoDiff framework

b.1)



(a) Example 1

(b) Example 2

Figure 1: Exercise b.1) graph

Figure 1a and 1b shows how we filled out the missing expressions for the different derivatives in the directed graphs representing example 1 and 2.

b.2)

After running

$$a = \text{Var}(3.0), \qquad b = \text{Var}(5.0), \qquad f = a \cdot b,$$

the created objects have the following internal values:

Variable a:

$$v = 3.0, \qquad \text{grad} = 0.0, \qquad \text{grad_fn} = [\,] \quad \text{(leaf node)}.$$

Variable b:

$$v = 5.0, \qquad \text{grad} = 0.0, \qquad \text{grad_fn} = [\,] \quad \text{(leaf node)}.$$

Variable $f = a \cdot b$:

$$v = 15.0, \qquad \text{grad} = 0.0,$$
$$\text{grad_fn} = [(a,\, b.v),\ (b,\, a.v)].$$

This corresponds to the partial derivatives of the multiplication:

$$\frac{\partial f}{\partial a} = b.v = 5, \qquad \frac{\partial f}{\partial b} = a.v = 3.$$

b.3)

For the first expression

$$f(a,b) = a \cdot b,$$
$$\frac{\partial f}{\partial a} = b, \qquad \frac{\partial f}{\partial b} = a.$$

Evaluated at $(a = 3)$ and $(b = 5)$:

$$\frac{\partial f}{\partial a}(3,5) = 5, \qquad \frac{\partial f}{\partial b}(3,5) = 3.$$

For the second expression

$$c = ab, \qquad e = ad, \qquad f = c + e,$$

the local partial derivatives are

$$\frac{\partial f}{\partial c} = 1, \qquad \frac{\partial f}{\partial e} = 1,$$
$$\frac{\partial c}{\partial a} = b, \qquad \frac{\partial c}{\partial b} = a,$$
$$\frac{\partial e}{\partial a} = d, \qquad \frac{\partial e}{\partial d} = a.$$

Using the chain rule:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial a} * \frac{\partial f}{\partial e}\frac{\partial e}{\partial a} = b + d,$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} = a,$$
$$\frac{\partial f}{\partial d} = \frac{\partial f}{\partial e}\frac{\partial e}{\partial d} = a.$$

Evaluated at $(a = 3)$, $(b = 5)$, $(d = 9)$:

$$\frac{\partial f}{\partial a} = 14, \qquad \frac{\partial f}{\partial b} = 3, \qquad \frac{\partial f}{\partial d} = 3,$$

and thus

$$\frac{\partial f}{\partial c} = 1, \qquad \frac{\partial f}{\partial e} = 1, \qquad \frac{\partial f}{\partial f} = 1.$$

b.4)

For the first expression

$$a = \mathrm{Var}(3.0), \qquad b = \mathrm{Var}(5.0), \qquad f = a \cdot b,$$

the call `f.backward()` triggers the following sequence of `backprop` calls:

1. `f.backprop(1.0)`
   - sets f.grad $\leftarrow$ f.grad $+ 1.0$
   - from f.grad_fn we get the pairs $(a,\ b.v)$ and $(b,\ a.v)$
2. `a.backprop(b.v * 1.0)`
   - sets a.grad $\leftarrow$ a.grad $+ b.v$
   - a.grad_fn is empty $\Rightarrow$ recursion stops for a
3. `b.backprop(a.v * 1.0)`
   - sets b.grad $\leftarrow$ b.grad $+ a.v$
   - b.grad_fn is empty $\Rightarrow$ recursion stops for b

So the actual call order is

`f.backprop(1.0)` $\rightarrow$ `a.backprop(b.v)` $\rightarrow$ `b.backprop(a.v)`.

For the second expression

$$c = ab, \qquad e = ad, \qquad f = c + e,$$

the call `f.backward()` triggers the following sequence of `backprop` calls:

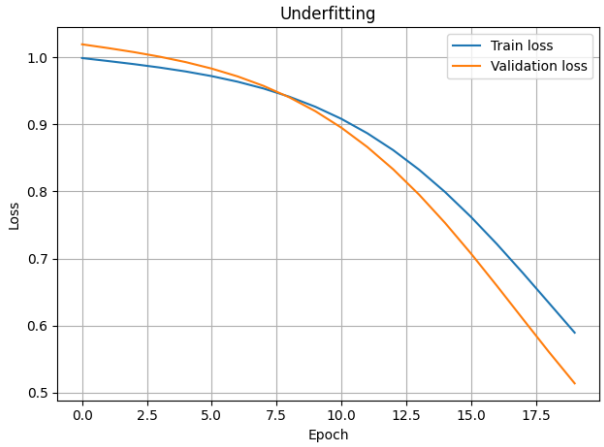
1. `f.backprop(1.0)`
   - sets f.grad $\leftarrow$ f.grad $+ 1.0$
   - from `f.grad_fn` we obtain the pairs $(c,\ 1)$ and $(e,\ 1)$.

2. `c.backprop(1.0)`
   - sets c.grad $\leftarrow$ c.grad $+ 1$
   - from `c.grad_fn` we obtain (a, b) and (b, a).
3. `a.backprop(b)`
   - sets a.grad $\leftarrow$ a.grad $+ b$
4. `b.backprop(a)`
   - sets b.grad $\leftarrow$ b.grad $+ a$
5. `e.backprop(1.0)`
   - sets e.grad $\leftarrow$ e.grad $+ 1$
   - from `e.grad_fn` we obtain (a, d) and (d, a).
6. `a.backprop(d)`
   - sets a.grad $\leftarrow$ a.grad $+ d$
7. `d.backprop(a)`
   - sets d.grad $\leftarrow$ d.grad $+ a$

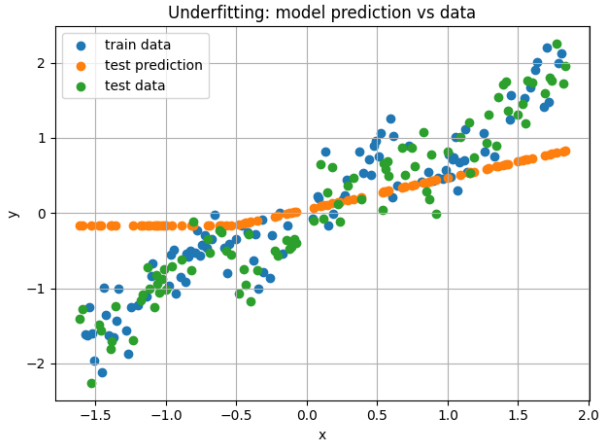
So the actual call order is

`f.backprop(1.0)` $\rightarrow$ `c.backprop(1.0)` $\rightarrow$ `a.backprop(b)` $\rightarrow$ `b.backprop(a)` $\rightarrow$ `e.backprop(1.0)` $\rightarrow$ `a.backprop(d)` $\rightarrow$ `d.backprop(a)`.

1)

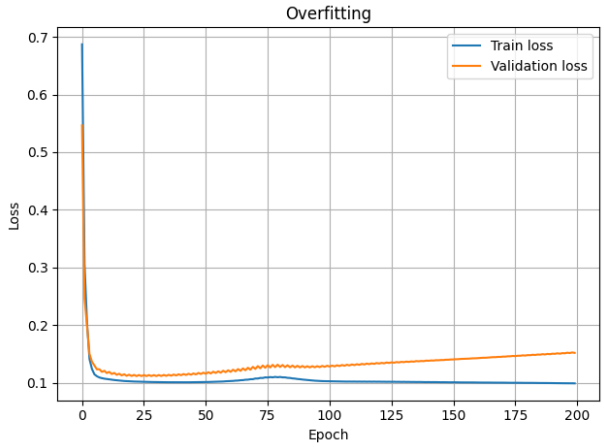


(a) MSE loss when underfitting.

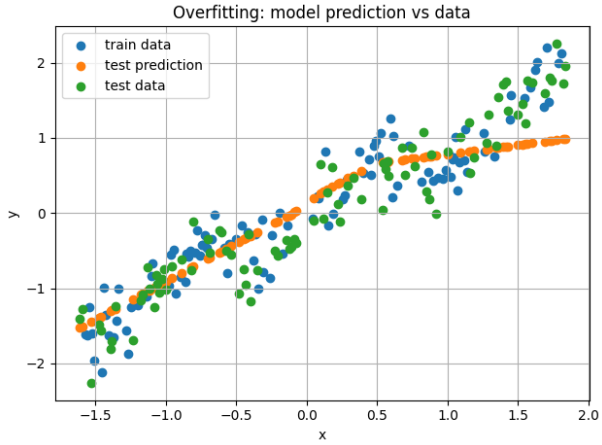


(b) Predictions when underfitting

Figure 2: Underfitting

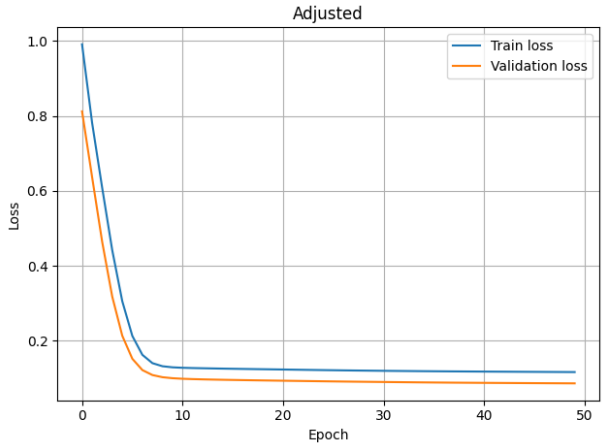


(a) MSE loss when overfitting.

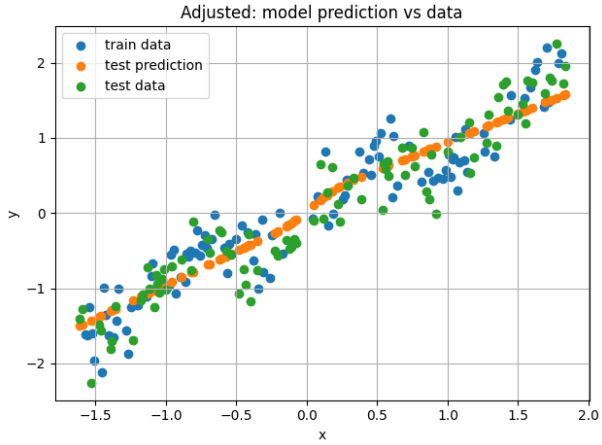


(b) Predictions when overfitting.

Figure 3: Overfit



(a) MSE loss when adjusted.



(b) Predictions when adjusted.

Figure 4: Adjusted

Underfitting.
The first experiment (see Figures 2a and 2b) shows clear underfitting. The model is too simple, consisting of only two small layers and trained for very few epochs. As a result, both the training and validation losses decrease slowly and remain relatively high, and the predictions collapse toward a nearly constant line rather than following the true relationship in the data. This behaviour reflects insufficient model capacity: the network does not have enough parameters to learn the nonlinear structure in the data. Because the architecture is extremely small and the training time is short, the model never reaches a level of complexity capable of capturing the pattern, making underfitting expected.

Overfitting.
The second experiment presented in Figures 3a and 3b shows overfitting. Here, the network was deliberately made large and was trained only on the first 75% of the data sorted by x. It is able to fit that portion extremely well, but once predictions extend into the unseen range, the model collapses toward almost constant outputs. This sharp mismatch between training performance and generalization is a hallmark of overfitting. The overfitting arises because the model capacity is far too high (250 hidden units), and it is trained for many epochs with an aggressively high learning rate. This combination causes the network to memorise the subset of data it sees rather than learning a stable mapping that generalizes beyond the training interval.

Adjusted fitting.
The final experiment shows an adjusted or "a balanced" fit (see Figures 4a and 4b). A moderate-sized network with 50 hidden units and a reasonable learning rate is trained for a moderate number of epochs. Here, training and validation losses follow a similar trajectory and remain low, and the predictions track both the training and test data well without collapsing or oscillating. This indicates that the model has sufficient but not excessive capacity. The chosen architecture prevents both extremes. It is large enough to capture the underlying relationship but not so expressive that it memorises noise or local patterns. The training duration and learning rate further help control complexity, resulting in stable and well-generalising behaviour.

Validation vs. test loss
For this particular dataset, there is no meaningful difference between validation and test loss. The data is i.i.d. and generated from the same underlying function, and both validation and test subsets cover the same input domain. Because of this symmetry, validation and test performance behave similarly.

3 Deep versus wide networks

3.1

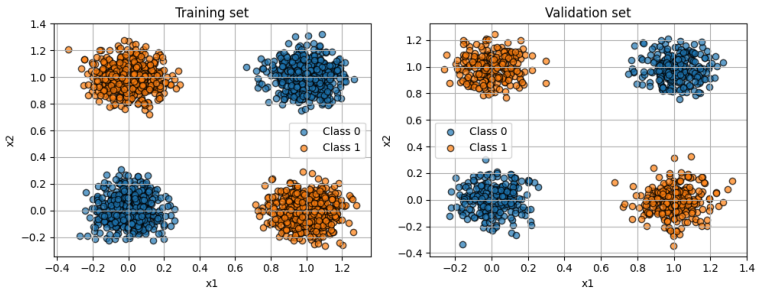


Figure 5: Training and validation sets

3.1.1 a

The first version of binary cross entropy loss function in pytorch is "BCELoss" which uses probabilities as inputs, the second version is "BCEWithLogitsLoss", which uses raw logits as inputs. To use "BCELoss" you need to use a Sigmoid layer on the output layer, where "BCEWithLogitsLoss" combines the Sigmoid layer with the "BCELoss" into one single class. The "BCEWithLogitsLoss" is more numerically stable than the first version, because combining the two operations into one, uses the log-sum-exp trick for numerical stability. We know from the description that the activation function of the output unit $\sigma()$ is built into the binary cross entropy loss function leading to the pytorch print out which shows that the model should output logits and the loss must apply the sigmoid layer internally. Therefore "BCEWithLogitsLoss" should be used.

3.2

3.3

Its clear from the table above that when depth is 0, the model performs poorly no matter how much width the model uses. The reason behind why the model is performing so badly with 0 depth, is because a linear network can not solve a XOR function. Increasing both depth and width increases the performance of the model, although it seems that having more depth than width is worse than them being equal.

Depth	Width	Mean Eval Loss	Std Eval Loss
0	1	0.693893	0.000620
0	2	0.692858	0.000613
0	3	0.693808	0.000694
1	1	0.478970	0.007145
1	2	0.207750	0.167753
1	3	0.070278	0.137632
2	1	0.483226	0.004355
2	2	0.001766	0.000966
2	3	0.000369	0.000279
3	1	0.476637	0.006723
3	2	0.170961	0.212096
3	3	0.000252	0.000195

Table 1: Evaluation loss statistics for depth/width combinations.

From chapter 4 in the book [**?**] we use that the number of weights in layer k is $\Omega_k = D_{k+1} \cdot D_k$ and the number of biases in layer k is $\beta_k = D_{k+1}$ then the number of parameters in layer k is:

$$D_{k+1} \cdot D_k + D_{k+1} = D_{k+1}(D_k + 1)$$

From the above function, we can derive a formula to calculate the number of parameters used for each combination of depth and width, first we derive the layer sizes for the network:

$$D_0 = 2, D_1 = W, D_2 = W, ..., D_D = W, D_{D+1} = 1$$

where the first layer is the input layer and the last in the output layer, there are D hidden layers each of width W. So the first layer, which is input layer to first hidden layer, is:

$$params_0 = D_1(D_0 + 1) = 3W$$

The next layer is hidden to hidden layers:

$$params_k = D_{k+1}(D_k + 1) = W^2 + W$$

with D hidden layers and the first hidden layer is already counted, it becomes:

$$\sum_{k=1}^{D-1} params_k = (D-1)(W^2 + W)$$

The last layer is from the last hidden layer to the output layer:

$$params_D = D_{D+1}(D_D + 1) = W + 1$$

Adding these three together gives us the function used to calculate the parameters for all the combination of depth and width in the network:

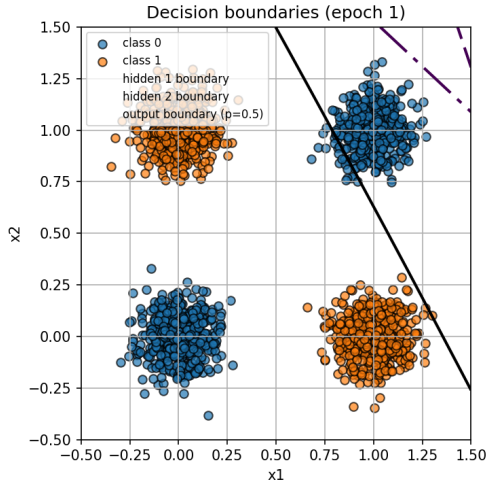
$$Params(D, W) = \begin{cases} 3, & D = 0 \\ 3W + (D-1)(W^2 + W) + (W+1), & D \geq 1 \end{cases}$$

Depth	Width	Number of Parameters
0	1	3
0	2	3
0	3	3
1	1	5
1	2	9
1	3	13
2	1	7
2	2	15
2	3	25
3	1	9
3	2	21
3	3	37

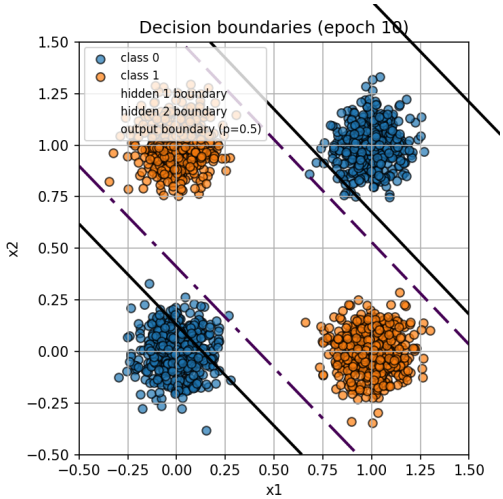
Table 2: Parameter counts for all tested networks

where D is depth and W is width. The table below shows how many parameters each of the tested networks has:
The initial hyperparameters are from creating the datasets, which are size of the training and validation set, another hyperparameter is noise level. These initial hyperparameters determine the difficulty of the learning problem. The hyperparameters of the data loader are batch size, which determines the number of training records used in forward and backwards pass, and shuffle which reshuffles the data for each epoch, thereby increasing robustness and helps avoiding under/overfitting. Epochs and learning rate are more hyperparameters used in the networks, the number of epochs determines how long the network trains, where deeper networks are negatively affected by undertraining, where shallow networks can appear better because of faster convergence. The learning rate affects fairness across depth and width, when the learning rate is too low, networks are undertrained, but a learning rate that is too high makes deep networks appear worse.

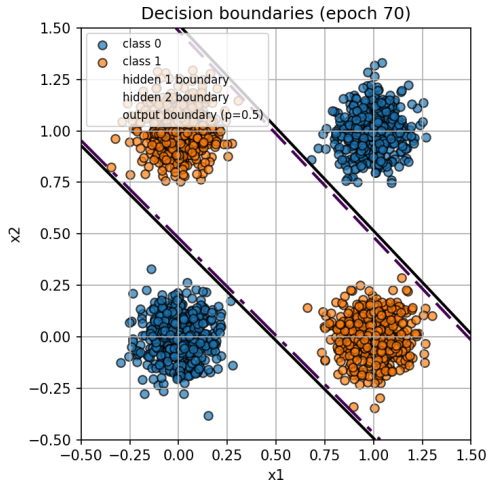
3.4



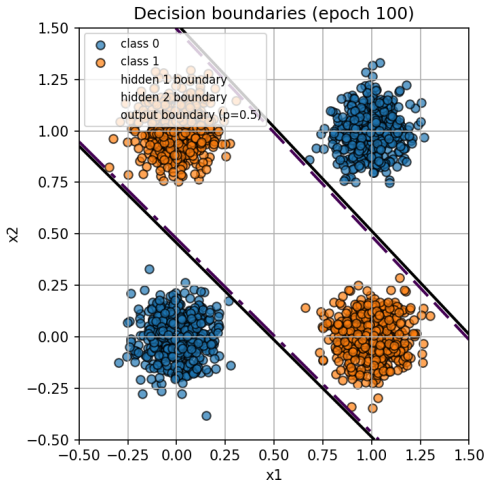
(a) First decision boundary



(b) Second decision boundary



(c) Third decision boundary



(d) Third decision boundary

Figure 6: Snapshots of decision boundaries

4 MLOps

4.1

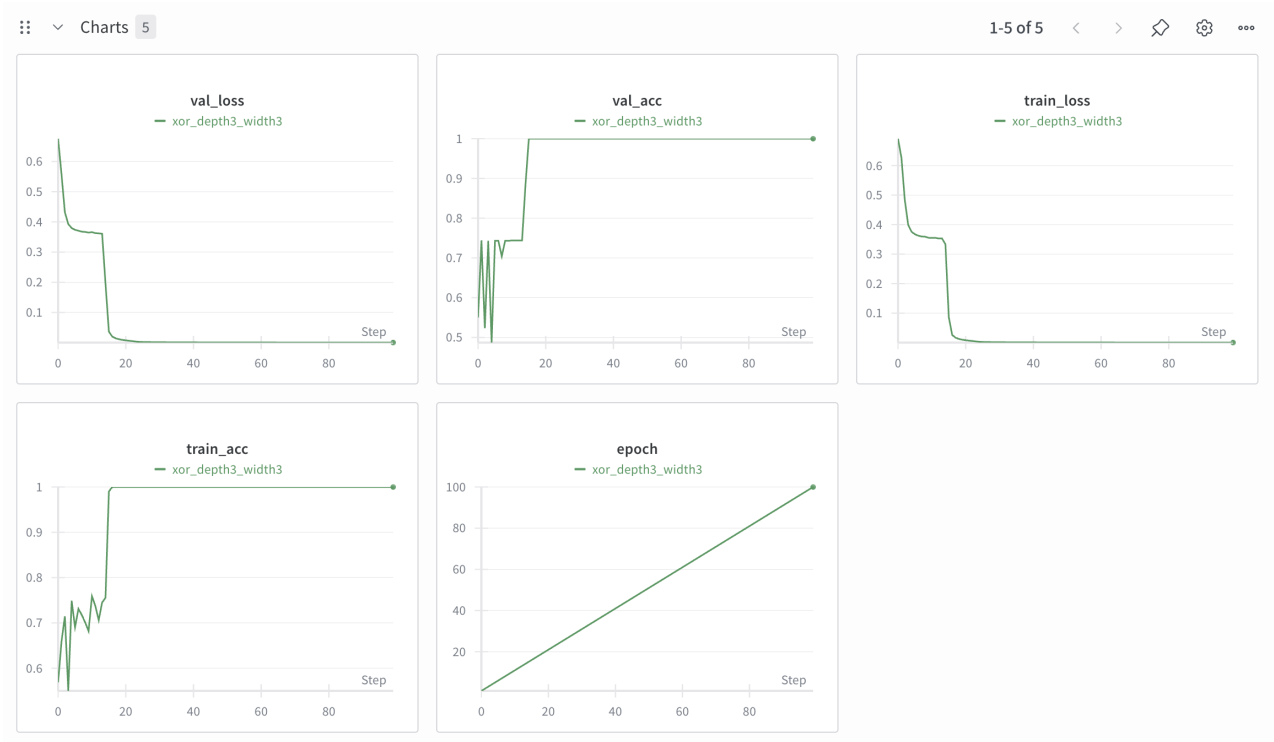


Figure 7: Logged training progress from W&B with set parameters.

In the code snippet below, it is shown how the function `train_model` is modified to log the training progress to W&B during each epoch. It sends the current epoch number, training/validation loss, and training/validation accuracy to the W&B dashboard, then stores the same values locally in the lists used to build the final history dictionary.

```
wandb.log(
            {
                "epoch": epoch,
                "train_loss": epoch_train_loss,
                "val_loss": epoch_val_loss,
                "train_acc": epoch_train_acc,
                "val_acc": epoch_val_acc,
            }
        )

        train_losses.append(epoch_train_loss)
        val_losses.append(epoch_val_loss)
        train_accuracies.append(epoch_train_acc)
        val_accuracies.append(epoch_val_acc)
```

In the next code snippet, it is shown how the code initializes a W&B run, records the main hyperparameters in the run's config, and tells W&B to track the model's gradients and parameter updates.

```
n_epochs = 100

wandb.init(
```

```
    project="dl-assignment1-xor",
    name="xor_depth3_width3",
    config={
        "model": "FeedForwardNet",
        "depth": 3,
        "width": 3,
        "n_train": n_train,
        "n_val": n_val,
        "batch_size": batch_size,
        "learning_rate": learning_rate,
        "n_epochs": n_epochs,
        "noise_std": s,
    },
)

wandb.watch(model, log="all", log_freq=10)
history = train_model(model, train_loader, val_loader, criterion,
    optimizer, n_epochs=n_epochs)
wandb.finish()
```

It then trains the network for `n_epochs` using `train_model`, logs all metrics automatically, and finally closes the W&B run. Note that we chose a network configuration with depth= 3 and width= 3, because these hyperparameters gave the lowest the evaluation loss statistics according to Table 1.

4.2

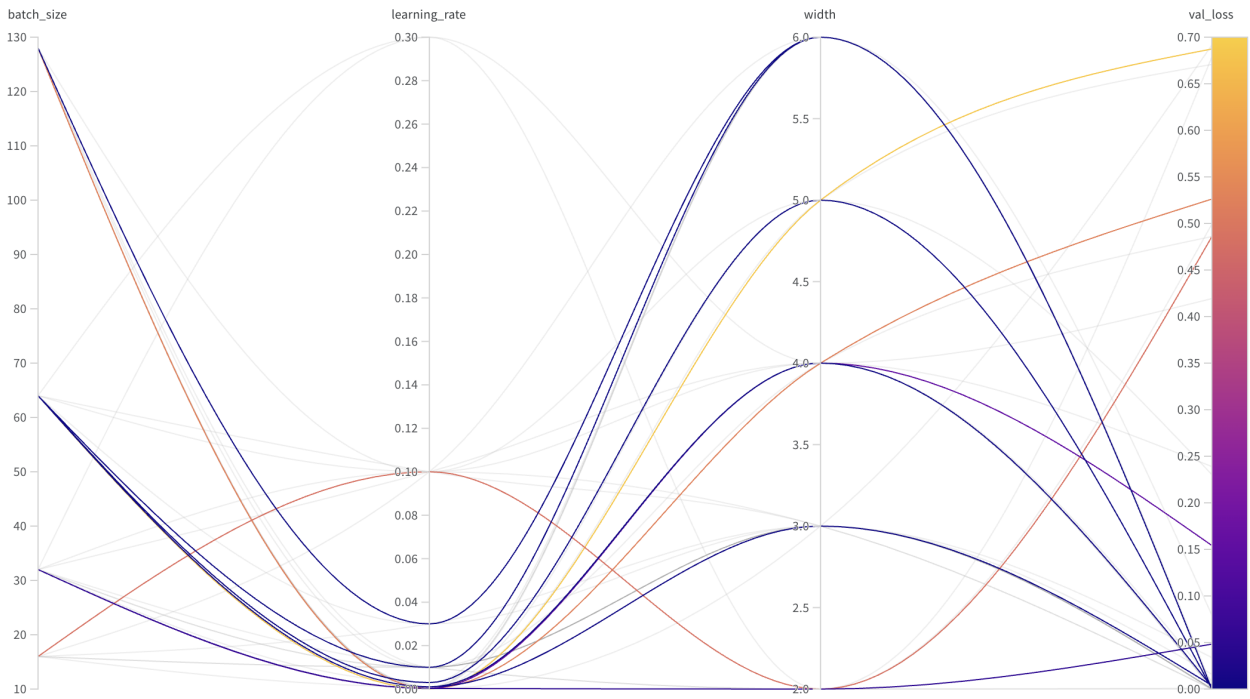


Figure 8: Sweep from W&B.

4.3

For the XOR problem, the most informative hyperparameters to tune are the learning rate, because optimization is highly sensitive to step size, and the network width and depth, which determine whether the model has enough non-linear capacity to represent the XOR structure. Batch size is also relevant, although mainly for optimization smoothness rather than model capacity.

However, we know that three hidden layers are sufficient because the depth–width experiment in the prior exercise clearly showed that networks with depth 3 achieve low validation loss on the XOR problem, indicating that they already have enough capacity to represent the non-linear decision boundary perfectly. Increasing the depth further does not improve performance on such a simple two-dimensional task and only adds unnecessary complexity, so we fix the depth at three in the sweep.

For the sweep experiment, each run trained the same feed-forward network used in Exercise 3, but with hyperparameters supplied by W&B's sweep agent. For every configuration, a new NoisyXORDataset was generated with fixed seeds to ensure comparability, and a model with depth three and a width determined by the sweep was initialized. Training used the existing 'train_model' function, which logs per-epoch loss and accuracy to W&B. Across runs, the sweep varied learning rate, batch size, and hidden-layer width, and W&B recorded all results for later comparison.

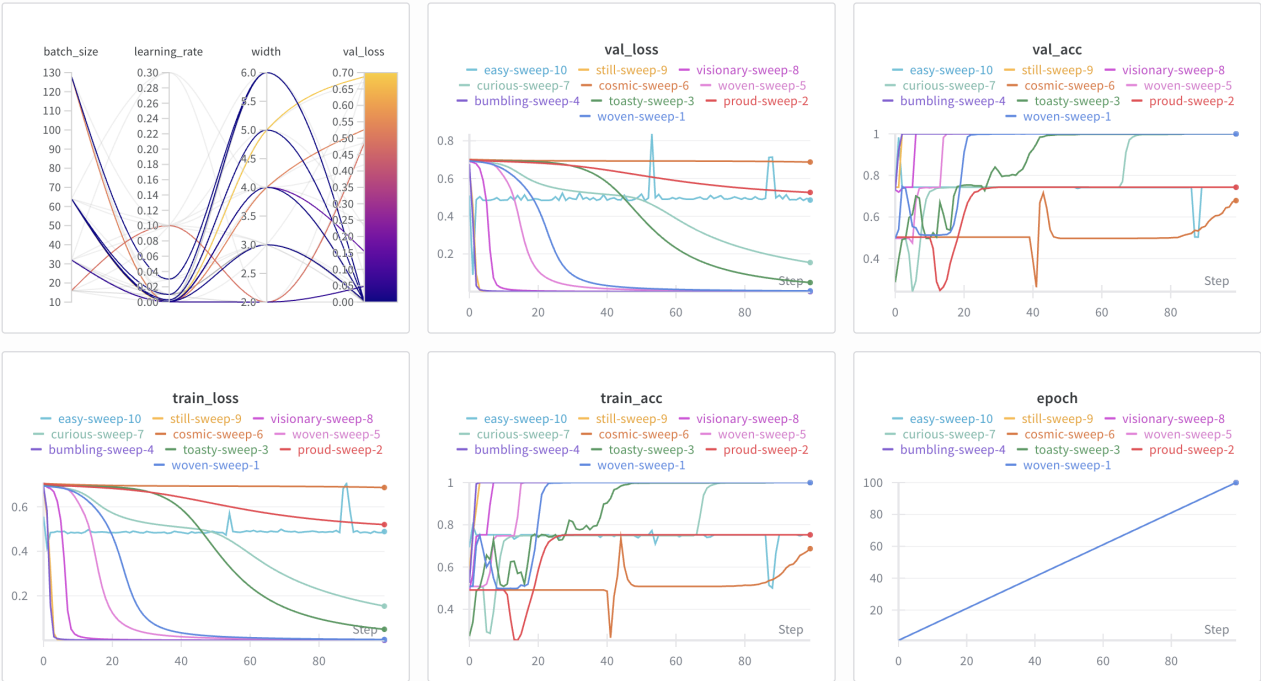


Figure 9: Results from the sweep on W&B.

The parallel coordinate plot and the parameter–importance table (see Figure 8 and Table 3) both indicate that learning rate and width are the dominant hyperparameters for this task, while batch size plays a smaller role. Lines with low validation loss in the parallel plot cluster around relatively small learning rates and mid-range widths, while extreme values on either axis tend to correlate with higher loss. This pattern is reinforced by the importance scores, which assign the highest influence to learning rate (0.585) and width (0.214), with batch size contributing the least (0.201). The comparatively low importance of batch size suggests it has limited influence on model generalization in this setup. Furthermore, the correlation values align with this ranking, showing that the learning rate is positively correlated with improved performance.

The plots in Figure 9 further support these findings. Runs with favourable learning rates and reasonable widths show smooth convergence in both train and validation loss, typically reaching high accuracy rapidly. In contrast, configurations with overly aggressive learning rates or extreme width values (either too shallow or too wide) tend to show plateauing or volatile loss curves. These trends validate the idea that for the XOR problem, optimization hyperparameters like the learning rate are more critical than capacity-related parameters like width or batch size. Together, Figures 8 and 9 demonstrate that while a minimum network width is necessary, successful training depends primarily on finding the right optimization regime.

In conclusion, the XOR task can be solved effectively by relatively small networks with moderate width and depth, combined with conservative learning rates and non-extreme batch sizes. Be aware that this is given that the experiment was done with very limited width and depth, to ensure convergence within a reasonable time frame. Given this setup different hyperparameter combinations all achieve very low validation loss, suggesting that the model class is more than expressive enough for the problem. Outside this experiment setup, training can still succeed but becomes slower and less stable, underlining that for simple problems like XOR, tuning optimization hyperparameters within a smaller hyperparameter space still can give reasonable results and that further tunning will probably lead to overfitting.

Config Parameter	**Importance**	**Correlation**
Learning rate	0.585	0.277
Width	0.214	0.009
Batch size	0.201	-0.164

Table 3: Hyperparameter importance and correlation based on W&B analysis w.r.t val loss.

Note. *W&B calculates importances using a tree-based model rather than a linear model, as the former is more tolerant of both categorical data and data that's not normalized.*